

Final Report

Group 21

Diogo Sousa fc59792

Bruno Faustino fc59784

Rodrigo Neto fc59850

June 13, 2026

Contents

1	Motivation	3
2	Dataset Characterization	3
3	Business Capabilities	3
3.1	Competitive Ranking	3
3.2	Custom Match Management	4
3.3	Custom Match History	4
3.4	Team Draft and Build Analyzer	4
4	Use Cases	4
4.1	Match & Ranking Management	4
4.2	Data Visualization & Analysis	4
5	API Specification	4
5.1	Player Management - Diogo Sousa	5
5.2	Queue Management - Diogo Sousa	5
5.3	Matchmaking - Diogo Sousa	6
5.4	Leaderboard - Diogo Sousa	6
5.5	Player Ranking - Diogo Sousa	6
5.6	Match History - Bruno Faustino	7
5.7	Match Filling - Bruno Faustino	7
5.8	Player Filling - Bruno Faustino	7
5.9	System - Rodrigo Neto	7
5.10	Training - Rodrigo Neto	8
5.11	Datasets - Rodrigo Neto	8
5.12	Games - Rodrigo Neto	9
5.13	Models - Rodrigo Neto	9
5.14	Features - Rodrigo Neto	9
5.15	Analysis - Rodrigo Neto	9
6	Functional Requirements	10
6.1	Matchmaking - Diogo Sousa	10
6.2	Ranking - Diogo Sousa	10
6.3	Match History - Bruno Faustino	10
6.4	Detail Filling - Bruno Faustino	11
6.5	Training - Rodrigo Neto	11
6.6	Data Analysis - Rodrigo Neto	11

7	Application Architecture	12
7.1	Matchmaking Service - Diogo Sousa	12
7.2	Ranking Service - Diogo Sousa	12
7.3	Match History Service - Bruno Faustino	12
7.4	Detail Filling Service - Bruno Faustino	13
7.5	Training Service - Rodrigo Neto	13
7.6	Analysis - Rodrigo Neto	13
7.7	Inter-Service Communication - Everyone	13
8	Non-Functional Requirements	13
8.1	Microservices Architecture - Diogo Sousa	13
8.2	Serverless & FaaS Integration - Bruno Faustino	14
8.3	DevOps & Observability	14
8.3.1	Observability Stacks - Bruno Faustino	14
8.3.2	Infrastructure as Code (IaC) & Automation	14
8.4	Security & Identity Management - Rodrigo Neto	15
8.4.1	Authentication Flow	15
8.4.2	Network & Bot Security	15
9	Technical Architecture	16
10	Deployment Scripts	16
10.1	Local Checks	16
10.2	Deployment	17
11	Tuning	17
12	CI/CD Pipeline	17
13	Test, Evaluation Techniques and Results (to target group)	17
13.1	Documentation Feedback - Diogo Sousa	17
13.2	Testing Results - Diogo Sousa	18
13.2.1	Smoke Testing (Endpoint: GET /api/listings/search)	18
13.2.2	Stress Testing (Endpoint: GET /api/listings)	18
13.3	Documentation Feedback - Bruno Faustino	18
13.4	Testing Results - Bruno Faustino	19
13.4.1	Smoke Testing (Endpoint: GET /api/auctions/{auction_id}/bids)	19
13.4.2	Stress Testing (Endpoint: GET /api/market/average_price)	19
13.5	Documentation Feedback - Rodrigo Neto	20
13.6	Testing Results - Rodrigo Neto	20
13.6.1	Smoke Testing (Endpoint: GET /api/market/price-comparison)	20
13.6.2	Stress Testing (Endpoint: GET /api/listings/stats/by-location)	20
14	Corrections Implemented after Feedback from Testing	20
15	Cost Analysis	21
16	Conclusions	21
 List of Figures		
1	Application Architecture	12
2	Technical Architecture	16

List of Tables

1	Match & Ranking Management Use Cases per Student	4
2	Data Visualization & Analysis Use Cases per Student	4
3	Player Management REST API	5
4	Queue Management REST API	5
5	Matchmaking REST API	6
6	Leaderboard REST API	6
7	Player Ranking REST API	7
8	Match History REST API	7
9	Match Filling REST API	7
10	Player Filling REST API	7
11	System REST API	8
12	Training REST API	8
13	Datasets REST API	8
14	Games REST API	9
15	Models REST API	9
16	Features REST API	9
17	Analysis REST API	10

1 Motivation

TODO

2 Dataset Characterization

- **Topic:** League of Legends Matches
 - **URL:** Kaggle - Matches Raw EUW
 - **Size:** Approximately 78 GB
 - **Release Date:** January 2026
-

3 Business Capabilities

3.1 Competitive Ranking

Facilitates real-time visualization of global and local leaderboards. Includes simplified performance metrics such as Damage dealt and KDA ratios.

3.2 Custom Match Management

Enables the creation of balanced custom matches. Matchmaking is handled via a global/local queue system integrated with the project's ranking logic.

3.3 Custom Match History

Provides a centralized repository for past custom match statistics. Includes robust functionality for custom filtering and sorting based on specific user requirements.

3.4 Team Draft and Build Analyzer

Offers personalized post-game analysis. Players utilizing the API receive automated insights regarding gameplay performance and actionable tips for improvement.

4 Use Cases

4.1 Match & Ranking Management

Use Case	Student Name
Leaderboards	Diogo Sousa
User Ranking	Diogo Sousa
Match Finding	Diogo Sousa
Create / Cancel Match	Diogo Sousa

Table 1: Match & Ranking Management Use Cases per Student

4.2 Data Visualization & Analysis

Use Case	Student Name
Match History	Bruno Faustino
Advanced History Filtering / Sorting / Paging	Bruno Faustino
Automatic Detail Filling (by interacting with RIOT's API)	Bruno Faustino
Draft Analysis	Rodrigo Neto
Build Analysis	Rodrigo Neto
Player Performance Analysis	Rodrigo Neto

Table 2: Data Visualization & Analysis Use Cases per Student

5 API Specification

5.1 Player Management - Diogo Sousa

Brief Description

Verb	Endpoint	Description
POST	/api/v1/matchmaking/ players	TODO
GET	/api/v1/matchmaking/ players/{id}	TODO
GET	/api/v1/matchmaking/ players/{id}/lobbies	TODO
GET	/api/v1/matchmaking/ players/{id}/tickets	TODO
DELETE	/api/v1/ranking/players/ links/{gameName}/ {tagLine}	TODO
GET	/api/v1/ranking/players/ {playerId}	TODO
PUT	/api/v1/ranking/ players/{playerId}/ link-lol-account	TODO
GET	/api/v1/ranking/ players/{playerId}/ primary-account	TODO
PUT	/api/v1/ranking/ players/{playerId}/ set-primary-account	TODO
POST	/api/v1/ranking/players/ {playerId}/sync-mmr	TODO

Table 3: Player Management REST API

5.2 Queue Management - Diogo Sousa

Brief Description

Verb	Endpoint	Description
POST	/api/v1/matchmaking/ queues	TODO
GET	/api/v1/matchmaking/ queues/{id}	TODO
PUT	/api/v1/ranking/queue/ {queueId}	TODO
GET	/api/v1/ranking/queue/ {queueId}	TODO
DELETE	/api/v1/ranking/queue/ {queueId}	TODO

Table 4: Queue Management REST API

5.3 Matchmaking - Diogo Sousa

Brief Description

Verb	Endpoint	Description
POST	/api/v1/matchmaking/tickets	TODO
POST	/api/v1/matchmaking/tickets/matches/{matchId}/complete	TODO
PUT	/api/v1/matchmaking/tickets/matches/{matchId}/link	TODO
POST	/api/v1/matchmaking/tickets/matches/{matchId}/players/{playerId}/accept	TODO
POST	/api/v1/matchmaking/tickets/matches/{matchId}/players/{playerId}/decline	TODO
DELETE	/api/v1/matchmaking/tickets/{ticketId}	TODO
GET	/api/v1/matchmaking/tickets/{ticketId}/lobby	TODO

Table 5: Matchmaking REST API

5.4 Leaderboard - Diogo Sousa

Brief Description

Verb	Endpoint	Description
GET	/api/v1/ranking/leaderboards	TODO
GET	/api/v1/ranking/leaderboards/queues/{queueId}	TODO

Table 6: Leaderboard REST API

5.5 Player Ranking - Diogo Sousa

Brief Description

Verb	Endpoint	Description
GET	/api/v1/ranking/ players/{playerId}/ queue-rankings	TODO
GET	/api/v1/ranking/ players/{playerId}/ queue-rankings{queueId}	TODO

Table 7: Player Ranking REST API

5.6 Match History - Bruno Faustino

Brief Description

Verb	Endpoint	Description
GET	/api/v1/history/matches	TODO
GET	/api/v1/history/matches/ {riotMatchId}	TODO

Table 8: Match History REST API

5.7 Match Filling - Bruno Faustino

Brief Description

Verb	Endpoint	Description
GET	/api/v1/riot/matches/ {matchId}	TODO
GET	/api/v1/riot/matches/ {matchId}/raw	TODO

Table 9: Match Filling REST API

5.8 Player Filling - Bruno Faustino

Brief Description

Verb	Endpoint	Description
GET	/api/v1/riot/players/ {server}/{name}/{tag}	TODO

Table 10: Player Filling REST API

5.9 System - Rodrigo Neto

Brief Description

Verb	Endpoint	Description
GET	/api/v1/analysis/	TODO
GET	/api/v1/analysis/health	TODO
GET	/api/v1/analysis/q/health/ready	TODO
GET	/api/v1/analysis/q/health/live	TODO
GET	/api/v1/training/	TODO
GET	/api/v1/training/health	TODO
GET	/api/v1/training/q/health/ready	TODO
GET	/api/v1/training/q/health/live	TODO

Table 11: System REST API

5.10 Training - Rodrigo Neto

Brief Description

Verb	Endpoint	Description
POST	/api/v1/training/jobs	TODO
GET	/api/v1/training/jobs	TODO
GET	/api/v1/training/jobs/{job_id}	TODO
POST	/api/v1/training/jobs/{job_id}/cancel	TODO
POST	/api/v1/training/jobs/{job_id}/deploy	TODO

Table 12: Training REST API

5.11 Datasets - Rodrigo Neto

Brief Description

Verb	Endpoint	Description
POST	/api/v1/training/datasets	TODO
GET	/api/v1/training/datasets	TODO
POST	/api/v1/training/datasets/build	TODO
GET	/api/v1/training/datasets/{dataset_id}	TODO
DELETE	/api/v1/training/datasets/{dataset_id}	TODO
POST	/api/v1/training/datasets/{dataset_id}/build	TODO

Table 13: Datasets REST API

5.12 Games - Rodrigo Neto

Brief Description

Verb	Endpoint	Description
POST	/api/v1/training/games	TODO
GET	/api/v1/training/games	TODO
POST	/api/v1/training/games/ batch	TODO
GET	/api/v1/training/games/ {game_id}	TODO
DELETE	/api/v1/training/games/ {game_id}	TODO

Table 14: Games REST API

5.13 Models - Rodrigo Neto

Brief Description

Verb	Endpoint	Description
GET	/api/v1/training/models	TODO
GET	/api/v1/training/models/ active	TODO
GET	/api/v1/training/models/ {model_id}	TODO
DELETE	/api/v1/training/models/ {model_id}	TODO
POST	/api/v1/training/models/ {model_id}/activate	TODO
POST	/api/v1/training/models/ {model_id}/deactivate	TODO

Table 15: Models REST API

5.14 Features - Rodrigo Neto

Brief Description

Verb	Endpoint	Description
POST	/api/v1/training/ features/extract	TODO
GET	/api/v1/training/ features/{game_id}	TODO
DELETE	/api/v1/training/ features/{game_id}	TODO

Table 16: Features REST API

5.15 Analysis - Rodrigo Neto

Brief Description

Verb	Endpoint	Description
POST	/api/v1/analysis/draft	TODO
POST	/api/v1/analysis/build	TODO
POST	/api/v1/analysis/player	TODO
POST	/api/v1/analysis/game	TODO
POST	/api/v1/analysis/ champion	TODO

Table 17: Analysis REST API

6 Functional Requirements

6.1 Matchmaking - Diogo Sousa

- Management of the matchmaking queue lifecycle, including ticket creation, status tracking, and cancellation;
- "On Join" matchmaking logic to evaluate and form lobbies immediately upon player entry;
- Implementation of a Standard Queue for 10-player lobby formation;
- Implementation of a Role Queue ensuring team balance across all five roles (TOP, JUNGLE, MID, BOT, SUPP);
- Lobby lifecycle control, including match acceptance/rejection and automatic lobby dissolution on failure;
- Coordination of match states from initial discovery to finalization and result reporting.

6.2 Ranking - Diogo Sousa

- Real-time calculation and updates of player skill ratings (ELO/MMR) based on match performance;
- Management of global and queue-specific leaderboards with paginated retrieval;
- Automatic player profile initialization and rank mapping via Riot account integration;
- Support for multiple ranking systems tailored to specific queues.

6.3 Match History - Bruno Faustino

- Local store of simplified match information relevant to the user (winning team, rank, roles of each player, number of kills, deaths, assists, healing, damage,..., per match, per team and per player), that can be inserted either manually, with validation, or by providing the match's ID in Riot's API;
- Match filtering based on any criteria the user defines from the simplified information, including obtaining all matches of a given player, all matches where the player had the most kills, ...;

- Match sorting based on any criteria;
- Match retrieval with paging to avoid fetching enormous amounts of data all at once;
- Soft deletion of matches from the history;
- New match data forwarding to the data analysers.

6.4 Detail Filling - Bruno Faustino

- Direct contact with Riot's API to obtain match and player information;
- Automatic filtering of information to focus more on the user's needs;
- Retrieval of the full raw data to be used for data analysis;
- Retrieval of player information (rank, wins, losses, points, ...).

6.5 Training - Rodrigo Neto

- Ingestion of raw match data in batches with automatic deduplication and metadata storage;
- Extraction and versioning of structured feature vectors for different analytical models;
- Creation and management of curated, optimized training datasets with configurable filtering;
- Asynchronous execution of ML training jobs for draft, build, and performance models;
- Real-time monitoring of training progress and lifecycle management of active jobs;
- Maintenance of a versioned model registry with performance metrics.

6.6 Data Analysis - Rodrigo Neto

- Win probability prediction for team compositions;
 - Identification of team synergies, counter-matchups, and win conditions;
 - Evaluation of item build effectiveness and possibly suggestion of alternatives;
 - Automated benchmarking of player performance metrics against global/rank-tier averages;
 - Global champion performance tracking including win, pick, and ban rate analytics.
-

7 Application Architecture

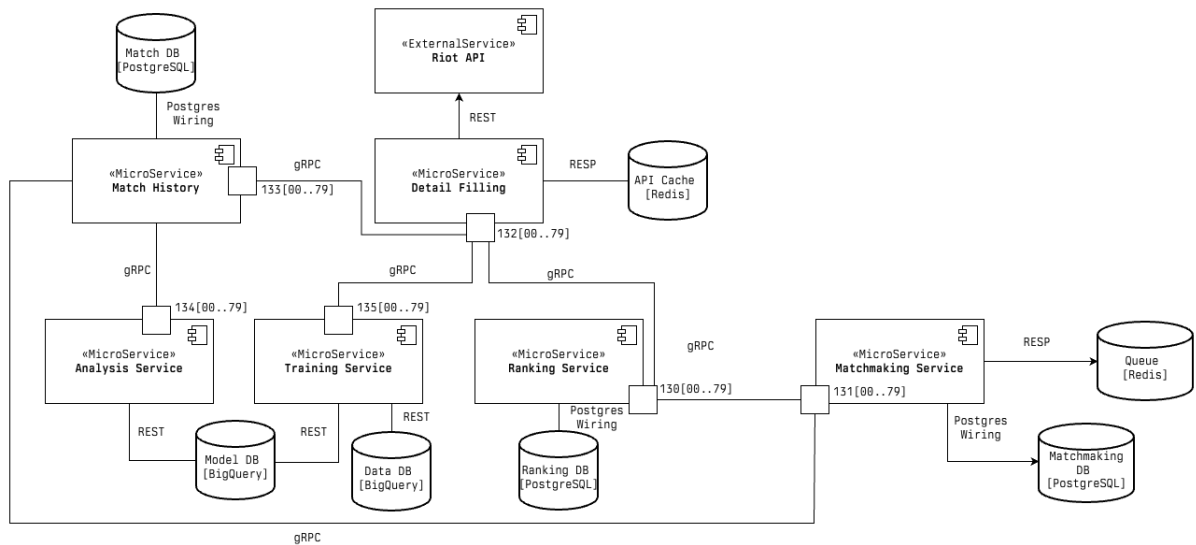


Figure 1: Application Architecture

7.1 Matchmaking Service - Diogo Sousa

- **Database: Redis (Queue/Locks) + PostgreSQL (Lobby Persistence)**
- **Usage:**
 - **Redis**: Manages the active matchmaking tickets and distributed locks for thread-safe lobby formation.
 - **PostgreSQL**: Stores Lobby, Match, and MatchTicket entities to track the lifecycle of a match from 'Searching' to 'Started' or 'Cancelled'.
- **Why**: **Redis** provides the **low latency** needed for high-frequency queue operations, while **PostgreSQL** ensures that lobby states and match history are **recoverable** even if a service pod restarts.

7.2 Ranking Service - Diogo Sousa

- **Database: PostgreSQL (Ranking DB)**
- **Usage**: Persists long-term player profiles, Discord/Riot account linkages, and MMR history. It stores the source of truth for a player's skill level and detailed win/loss/stat records across various queues.
- **Why**: Requires **ACID compliance** to ensure that MMR updates and account linking are handled atomically and reliably.

7.3 Match History Service - Bruno Faustino

- **Database: PostgreSQL (History DB)**
- **Usage**: Indexes detailed match statistics for players and teams, supporting complex filtering (e.g., 'all matches where player X had the most kills').

- **Why: Relational PostgreSQL** is ideal for complex queries, sorting, and paginated views of historical match data.

7.4 Detail Filling Service - Bruno Faustino

- **Database: Redis (API Cache)**
- **Usage:** Caches Riot API responses for accounts and match details.
- **Why:** Essential for **performance** and **cost-efficiency**; it minimizes calls to the external Riot API, preventing rate-limiting/API key issues and speeding up data retrieval for other services.

7.5 Training Service - Rodrigo Neto

- **Database: BigQuery (Model DB)**
- **Usage:** Responsible for training and storing machine learning models using historical match data.
- **Why:** Separating training from analysis ensures that **resource-intensive model training** does not compete with real-time insight requests.

7.6 Analysis - Rodrigo Neto

- **Database: BigQuery (Data DB)**
- **Usage:** Consumes match data to provide real-time and post-game insights, such as draft analysis and player performance metrics.
- **Why:** BigQuery allows for **efficient querying of massive datasets** without impacting the performance of the operational transactional databases.

7.7 Inter-Service Communication - Everyone

- The services communicate reliably through **gRPC** for high-performance internal calls (e.g., Matchmaking asking Ranking for a player's MMR) and **REST** for external integration and management.

8 Non-Functional Requirements

8.1 Microservices Architecture - Diogo Sousa

The core architecture focuses on high availability and loose coupling through message-driven patterns.

Data & Communication

- **Database Replication:** Implementation of replication strategies to ensure data persistence and availability across microservices.

- **Message-Driven:** Utilization of **RabbitMQ** for asynchronous, decoupled communication.
- **Orchestration:** Deployed within **Kubernetes** for scalable message processing.
- **Integration:** Support for **Quarkus** (Java) and **Python** clients to facilitate real-time data processing.

8.2 Serverless & FaaS Integration - Bruno Faustino

To optimize resource usage, specific background tasks (such as detail filling) will transition to a Serverless model using **Google Cloud Functions**.

- **Implementation:** **Quarkus Funqy** for a portable, provider-agnostic function-as-a-service (FaaS) experience.
- **Communication:** Utilizing **gRPC** for high-performance communication between the serverless functions and existing microservices.

8.3 DevOps & Observability

A multi-layered approach to infrastructure management and system monitoring.

8.3.1 Observability Stacks - Bruno Faustino

We are thinking about two primary observability stacks for monitoring metrics, logs, and traces.

Option A: The LGTM Stack (Grafana Cloud)

- **Loki:** Log aggregation system using label-based indexing for cost-efficiency.
- **Grafana:** Central visualization hub for dashboards and alerting.
- **Tempo:** High-scale distributed tracing backend.
- **Mimir/Prometheus:** Long-term storage and collection of time-series metrics.
- **Alloy:** OpenTelemetry-native collector for forwarding metrics, logs and traces data.

Option B: VictoriaMetrics Stack

- **VictoriaMetrics:** Handles metrics and alerts via MetricsQL.
- **VictoriaLogs:** Dedicated log database using LogsQL.
- **VictoriaTraces:** Dedicated tracing backend.
- **Ingestion:** Native **OpenTelemetry** support for all telemetry types.

8.3.2 Infrastructure as Code (IaC) & Automation

8.3.2.1 GKE Automation - Rodrigo Neto

- **GKE Automation:** Provisioning Google Kubernetes Engine (GKE) using **Terraform** or **Pulumi** (Python).

8.3.2.2 Configuration Management & Continuous Delivery - Bruno Faustino

- **Configuration Management:** Using **Ansible** to manage Kubernetes ConfigMaps and software state.
- **Continuous Delivery:** **Argo CD** will be used for GitOps, ensuring the cluster state automatically reflects the Git repository via pull requests.

8.4 Security & Identity Management - Rodrigo Neto

Security is enforced at both the application level (JWT) and the network level (Service Mesh).

8.4.1 Authentication Flow

The Authentication Microservice (AMS) manages the user lifecycle:

1. **Account Creation:** AMS validates credentials and stores the user hash in a database.
2. **Authentication:** User logs in, AMS validates credentials, generates a **JWT**, and stores it in a **Redis** cache for session management.
3. **Request Validation:** The user provides the JWT in request headers. Microservices validate the token with AMS (or via Service Mesh policy) to grant access.

8.4.2 Network & Bot Security

- **Istio Service Mesh:** Enforces **mTLS** (mutual TLS) for secure service-to-service communication and decentralized authorization.
 - **Discord Bot Validation:** Action validation between the Discord bot and backend services is secured via a shared secret.
 - Requests are encrypted with the shared secret, including an account ID and timestamp to prevent replay attacks.
-

9 Technical Architecture

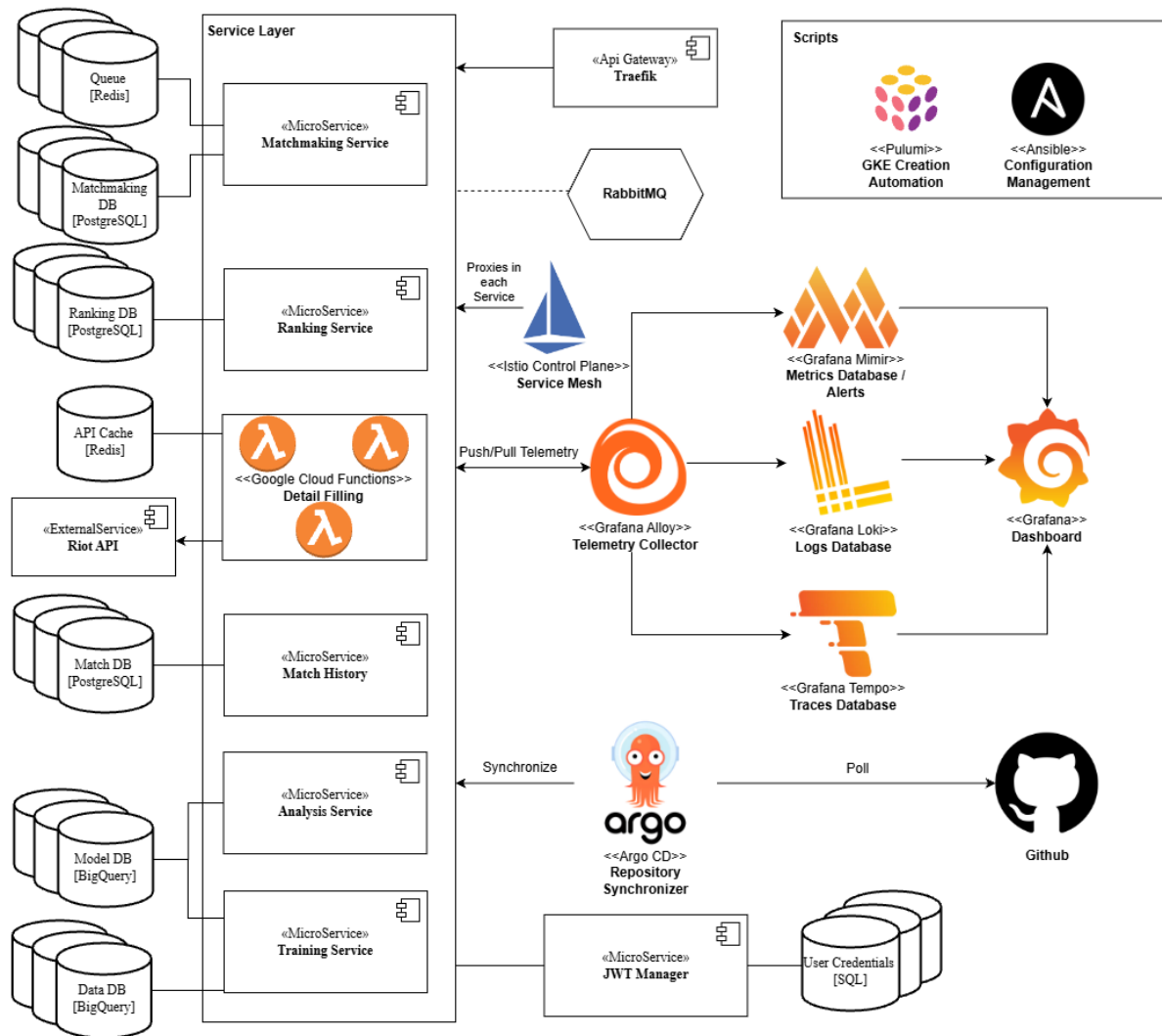


Figure 2: Technical Architecture

10 Deployment Scripts

10.1 Local Checks

```
./scripts/format.sh
./scripts/test.sh
```

Python gRPC files are generated, not committed:

```
./scripts/python/generate-grpc.sh
```


10.2 Deployment

Required environment:

```
export SCRIM_PROJECT_ID="your-gcp-project "  
export SCRIM_REGION="europe-west1 "  
export SCRIM_REPO_NAME="scrimfinder "  
export SCRIM_CLUSTER_NAME="scrimfinder "  
export RIOT_API_KEY="RGAPI-..."  
export SCRIM_DB_USER="scrim_admin "  
export SCRIM_DB_PASSWORD="secure_password "
```

Deploy infrastructure, images, Cloud Functions, and the Helm application:

```
./scripts/deploy.sh
```

Tear down the stack:

```
./scripts/shutdown.sh
```

11 Tuning

TODO - cpu/memory requests and limits, hpa, vpa, liveness and readiness checks with initialDelay/period/timeout

12 CI/CD Pipeline

TODO - on push to main and work/*, on pull-request to main, test types of each and brief description of the tests, ...

13 Test, Evaluation Techniques and Results (to target group)

13.1 Documentation Feedback - Diogo Sousa

- **Style Discrepancy:** The query parameter for pagination size is documented as `pageSize` (camelCase) while the response object uses `page_size` (snake_case). The API Gateway correctly accepts the `pageSize` parameter and returns the corresponding schema, but this inconsistency should ideally be unified.
- **Error Response Documentation:** While standard HTTP error codes (400, 401, 404, 500, 502) are consistently mapped across endpoints using reusable components (e.g.,

`BadRequestText`), they could be more descriptive. The schema indicates they return a `text/plain` string without explaining *what* triggers them (e.g., what specific validation failure causes a 400, or under what conditions a 404 is thrown for a specific endpoint).

13.2 Testing Results - Diogo Sousa

13.2.1 Smoke Testing (Endpoint: [GET](#)/api/listings/search)

The smoke test was executed using a **Python script** checking the endpoint. The test validated the HTTP 200 response, JSON validity, and the presence of all required top-level fields (`total`, `page`, `page_size`, `listings`) and listing level attributes.

- **Basic Paginated Search:** Passed. Returned 5 listings with a reported total of 50,000 records. Latency: $\approx$ 141ms.
- **Filtered Search (Toyota Corolla):** Passed. Returned 5 listings with a reported total of 300 records. Latency: $\approx$ 191ms.

No broken links or malfunctioning behaviors were detected.

13.2.2 Stress Testing (Endpoint: [GET](#)/api/listings)

The stress test was executed using **Locust** against endpoint.

- **Concurrency Settings:** 100 concurrent users, spawning at a rate of 20 users per second, run duration of 30 seconds.
- **Performance observed:**
 - **Total Requests:** 6,899
 - **Failures:** 0 (0.00%)
 - **Requests Per Second (RPS):** Peaked at $\approx$ 275 req/s (average 248.81 req/s over the duration).
 - **Latency Statistics:** Average = 72ms, Median = 62ms, 95th percentile = 93ms, 99th percentile = 150ms.

The endpoint proved to be highly stable under the tested load.

13.3 Documentation Feedback - Bruno Faustino

- **Some positives:**
 - The API documentation was in general easy to understand due to the modularity on the groups of endpoints and the descriptive information tagged with each endpoint.
 - The introductory paragraph gives a quick overview of all the use cases of the application and in which modules they can be found.
 - In general, the exceptions portray in which cases the app may fail (although they could be explained in more detail according to the endpoint in some cases).
- **A few suggestions:**
 - For the chat endpoints, a 404 code should be thrown if the chat id does not exist in the database.

- It would be helpful to have default values already filled-in in the body of the POST methods that serve as an example to further improve the understandability of the API (for some fields it is not trivial what the expected format is, like `fuel_type`, `drive_type` and `end_time`).
- Although it is well explained that the user needs to add the Host header before calling the post/put/delete endpoints, it is never explained that the user needs to be logged in and that the resulting token needs to be added to the header: a quick mention in either the introductory paragraph or the 401 Unauthorized exception in the documentation would generate less confusion and save a few 401s.

13.4 Testing Results - Bruno Faustino

13.4.1 Smoke Testing (Endpoint: `GET/api/auctions/{auction_id}/bids`)

The smoke test was executed using **pytest** and **requests** modules of **Python**.

Here is the sequence of API calls as a setup before testing the actual endpoint:

1. **POST**/`api/user/login` for the auth token.
2. **POST**/`api/listings` for `listing_id`.
3. **POST**/`api/auctions` for `auction_id`.
4. 3x **POST**/`api/auctions/{auction_id}/bid`.

After calling the endpoint: `GET/api/auctions/{auction_id}/bids`, it is expected that the command returns successfully (200) and with 3 auction bids in the created auction for the created listing.

The smoke test passed!

13.4.2 Stress Testing (Endpoint: `GET/api/market/average_price`)

The stress test was executed using **Locust** against the endpoint.

- **Concurrency Settings:** 10000 concurrent users, spawning at a rate of 5 users per second, running until failing.
- **Performance observed:**
 - **Total Requests:** 250
 - **Failures:** 4 (1.60%)
 - **Requests Per Second (RPS):** Very inconsistent, around 19.61 req/s.
 - **Latency Statistics:** Average = 1009ms, Median = 190ms, 95th percentile = 4000ms, 99th percentile = 6700ms.

The system fails a few times at 19.61 req/s due to the unavailability of an upstream service (502).

During other rounds of testing, the system could fail at less or more than that value, sometimes managing to survive without failures until around 50 req/s, so it is inconsistent, and the number of failures oscillates a lot between load tests, up to the point of reaching 99% failures.

The endpoint proved unable to sustain loads above 50 req/s, and shows signs of failing around 19.61 req/s.

13.5 Documentation Feedback - Rodrigo Neto

- **Missing Content-Type Header:** Successful JSON responses do not include the `Content-Type: application/json` header, which can break strict HTTP clients.
- **Poor Overload Behaviour:** Under high load, the backend does not gracefully degrade (e.g., return 429 or 503). Instead, the gateway returns 502 `Bad Gateway` after extremely long timeouts (up to 47 seconds).

13.6 Testing Results - Rodrigo Neto

13.6.1 Smoke Testing (Endpoint: [GET/api/market/price-comparison](#))

The smoke test was executed using **Postman** checking the endpoint. The test validated HTTP 200 responses, JSON validity, pagination, grouping, and sorting behaviors.

- **Happy Path:** Passed with warnings. Valid JSON, correct pagination and sorting. `Content-Type` assertion failed (header omitted).
- **Validation:** Partial pass (3/6). Missing parameters returned 400. Invalid enum values returned 502 `Bad Gateway` instead of 400.

Some malfunctioning behaviors were detected during validation tests.

13.6.2 Stress Testing (Endpoint: [GET/api/listings/stats/by-location](#))

The stress test was executed using **Locust** against the endpoint.

- **Concurrency Settings:** Step-load ramping from 10 to 200 concurrent users.
- **Performance observed:**
 - **Total Requests:** 3,524
 - **Failures:** 123 (3.49%)
 - **Requests Per Second (RPS):** Peaked at ≈ 50 req/s.
 - **Latency Statistics:** Average = 7,280ms, Median = 460ms, 95th percentile = 34,000ms, 99th percentile = 38,000ms.

The endpoint proved unable to sustain loads above ≈ 50 RPS, leaving the upstream backend in an unhealthy state.

14 Corrections Implemented after Feedback from Testing

TODO

15 Cost Analysis

TODO

16 Conclusions

TODO
